



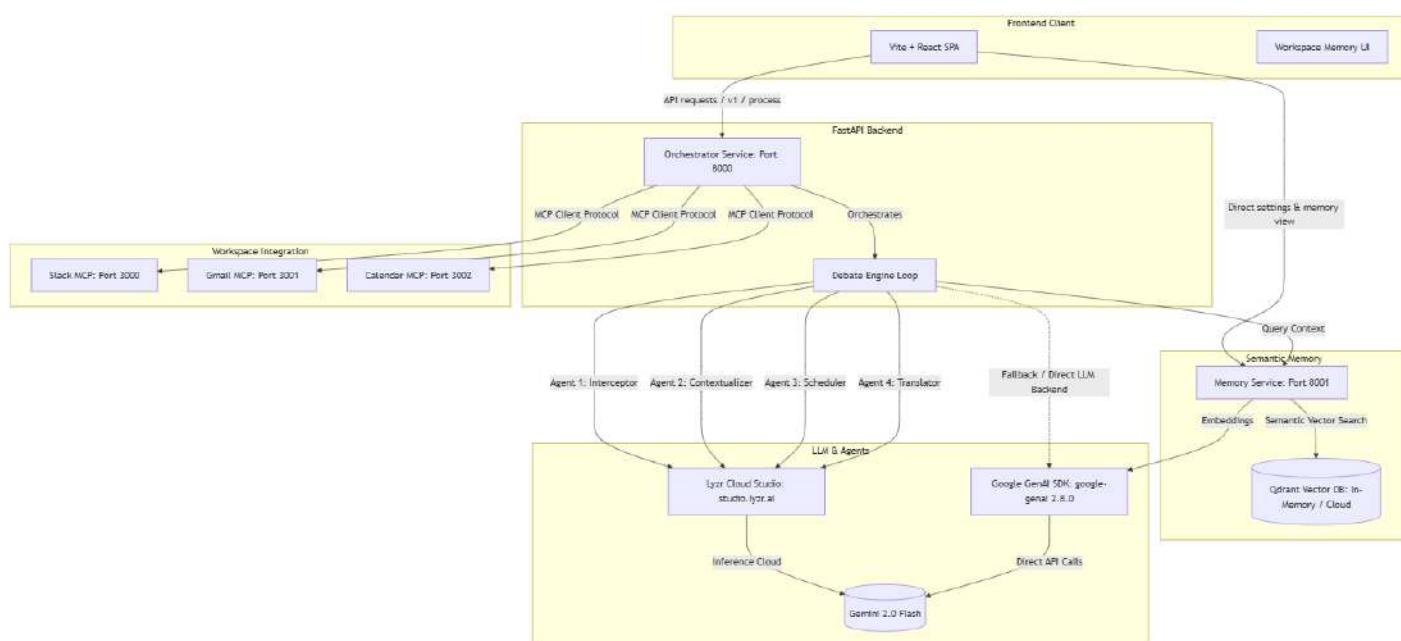
Workplace Proxy (Project Clarity) — Hackathon Technical Blueprint

Workplace Proxy (Project Clarity) is an AI-powered assistive communication proxy designed for neurodivergent professionals. It intercepts vague, ambiguous, or stress-inducing workplace messages (from Slack, Gmail, etc.) and translates them into structured, explicit, and anxiety-reducing tasks.

This architecture leverages a **Multi-Agent Debate Engine** powered by **Lyze-ADK**, retrieves cognitive and corporate memory via **Qdrant Vector Database**, integrates with workspace tools using **Model Context Protocol (MCP)**, and maintains enterprise-level resilience using the **Google GenAI SDK**.



Core Architectural Overview

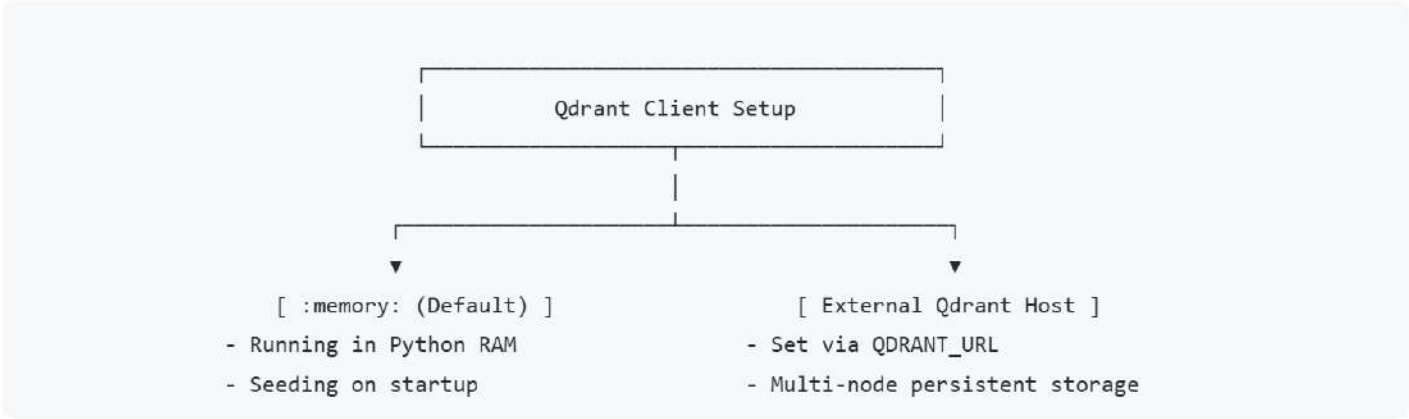


Qdrant: Semantic Memory & Context Alignment

Qdrant acts as the long-term cognitive memory for the proxy. It allows the agent pipeline to perform semantic search over user preference patterns and corporate rules before executing the final translation.

1. Database Topology

Qdrant is run **in-process in-memory** by default for zero-config hackathon execution, but seamlessly supports external persistent clusters (`QDRANT_URL`).



2. Schema Definition (768 Dimensions)

The vector space is aligned to Google’s `text-embedding-004` (768 dimensions). The system maintains three specialized collections:

Collection	Payload Schema	Search Filters & Fallbacks
user_context	formatting_style , preferred_urgency_language , working_hours_start , working_hours_end , deep_work_blocks , known_triggers	Filtered strictly by user_id
corporate_context	relevant_projects , jargon_decoded (JSON), sender_history , relevant_docs	Unfiltered search + Sender-bias boost
feedback	rating (1-5), agent_outputs , original_message	Post-execution evaluation logs

3. Smart Search & Decoupling Mechanics

To resolve jargon, the system runs two concurrent vector search passes:

- Direct Message Query:** Matches names of projects, databases, and onboarding paths.
- Sender Boost Search:** If a sender name is identified (e.g. "Alice"), a second vector search boosts documents tagged with Alice's historical communication behavior (e.g., "Alice says 'no rush' but means EOD").

Lyzr ADK: Agentic Design & Cloud Orchestration

The project utilizes `lyzr-adk >= 0.1.11` to handle the instantiation, configuration, and execution of agent personas.

1. Multi-Agent Debate Engine Configuration

Workplace Proxy defines four distinct agent personas. In the production deployment, these can be set in two configurations controlled via `.env` :

- Shared Backend (Default):** A single multi-agent cloud orchestrator (`project_clarity_orchestrator`) handles all calls, with role instructions injected on the fly.
- Per-Agent Mode (`LYZR_PER_AGENT=true`):** Creates four separate agent IDs in Lyzr Studio with isolated telemetry.

Lyzr-ADK Agent Setup			
Agent Name	Role	Goal	Persona
Interceptor	Message Analyst	Structural analysis	Extract vague phrases
Contextualizer	Context Decoder	Decode jargon	Match corporate facts
Scheduler	Temporal Planner	Enforce realism	Convert terms to times
Translator	Clarity Spec.	Clear task rewrites	Format final translation

2. Python Initialization Flow

```
# Create the studio client using the API key
studio = Studio(api_key=settings.lyzr_api_key)

# Instantiate the Agent with a Gemini-backed provider
agent = await studio.create_agent(
    name="clarity_translator",
    description="Neurodivergent Communication Specialist",
    role="Clarity Communication Specialist",
    provider="google/gemini-2.0-flash",
    llm_credential_id="lyzr_google",
    store_messages=False, # Keeps the execution stateless
    temperature=0.3
)
```

Google GenAI SDK: Direct Execution & Enterprise Resilience

When direct SDK capabilities or lower latency are required (or when Lyzr is disabled), the system bypasses cloud agents and speaks directly to Gemini 2.0 via the `google-genai` SDK.

1. Features Utilized

Structured Output (JSON Schema): Forcing agents to return exact, typed payloads matching Pydantic definitions.

Gemini 2.0 Flash (`gemini-2.0-flash`): Used for quick reasoning and low response latency.

Google Embeddings (`text-embedding-004`): Powers semantic calculations for Qdrant.

2. Robust Retry Loop (Exponential Backoff)

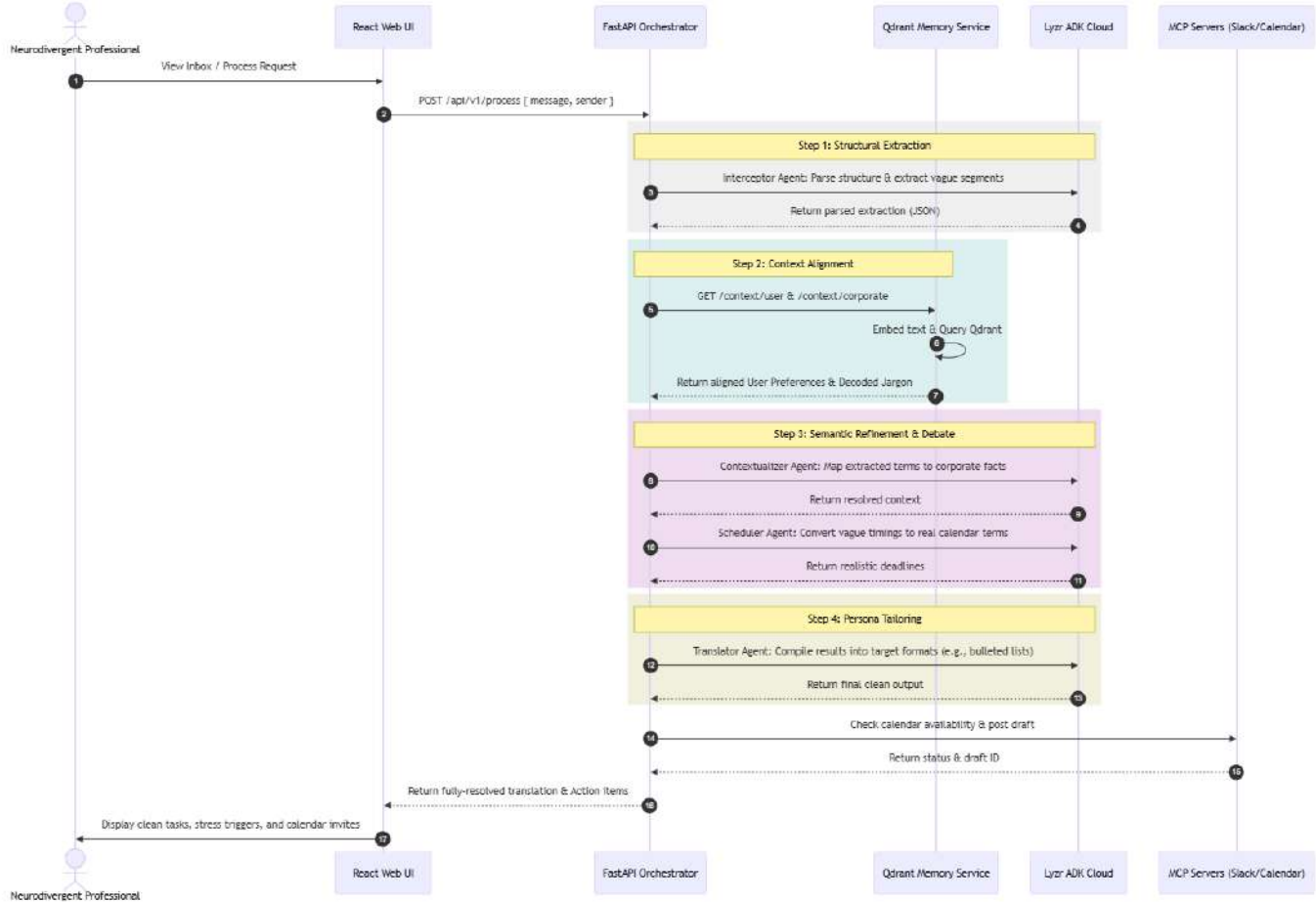
To withstand transient network issues or API rate limiting during high-stress live hackathon presentations, direct SDK calls are wrapped in a resilient runner:

$$\text{Delay} = \text{retry_delay} \times 2^{\text{attempt}}$$

```
def _call_with_retry(self, fn: Any, *args: Any, **kwargs: Any) -> Any:
    last_exc = None
    for attempt in range(self._retry_count + 1):
        try:
            return fn(*args, **kwargs)
        except Exception as exc:
            last_exc = exc
            if attempt < self._retry_count:
                delay = self._retry_delay * (2 ** attempt)
                time.sleep(delay)
    raise last_exc
```

Dynamic Pipeline Dataflow

This sequence diagram displays how an incoming message traverses the system:



Hackathon Execution Guide

Fast Startup (Docker-Free Mode)

Install Prerequisites: Ensure you have Python 3.11+ and Node.js 18+.

Setup Envs: Copy files and fill in credentials.

```
cp .env.example .env
cp backend/.env.example backend/.env
```

Start All Services: Run the service launcher script:

```
node start-services.cjs
```

This command starts the Frontend (5173), Orchestrator (8000), Memory Service (8001), and mock MCP providers (3000-3002).

[!TIP] **Observation Dashboard:** Visit <http://localhost:5173/settings> during execution to view active agent backends and see how vectors are bound dynamically in memory.